

Premier Receiver Automation — API Flow

Every endpoint between a delivery email and a receipt in sfPMS. Responses captured live from <https://training.remingtonhotels.com/Training> on 2026-08-18.

Nothing in this document is quoted from a specification. Every response shown was returned by the training server and can be regenerated with `tools/verify_prd_endpoints.py`, which calls each endpoint again and rewrites the capture this document is built from.

The one thing to understand first

Spitfire cannot tell you what you have already posted. `ReceiptInProgressUnits` — the one field that looks like it would answer “has this delivery been received?” — reads **0.0** for the whole time a receipt sits unapproved, which is the entire window that matters. Nothing in this API is idempotent, and the document catalog does not deduplicate identical bytes. So every guard against double-posting lives in our own ledger, not in the ERP.

Measured against PO 212559 on 2026-08-18. Lines 0001, 0002 and 0004 carry receipts we posted the same morning; the ERP reports nothing on any of them:

Line	Spec	ContractUnits	UOM	ItemQuantity	ReceivedUnits	ReceiptInProgressUnits
0001	BRR-803-AC	4.0	Set	0.0	0.0	0.0
0002	FRR-803-AC	4.0	Set	0.0	0.0	0.0
0003	—	0.0	—	0.0	0.0	0.0
0004	PRR-803-AC	2.0	Set	0.0	0.0	0.0
0005	LRR-803-AC	2.0	Set	0.0	0.0	0.0
Freight	—	0.0	—	0.0	0.0	0.0

Two further traps visible in that table: `ItemQuantity` reads **0.0** on every line — the ordered quantity is `RelatedLineDetails.ContractUnits` — and the spec code lives in `SourceItemNumber`, because `Specification` is null on real purchase orders.

Step 1 — a mail arrives

Microsoft Graph, application permissions (`Mail.ReadWrite`), restricted to the one mailbox by an Application Access Policy. Auth is a bearer token from MSAL client-credentials — not a cookie; that is Spitfire's mechanism, not Graph's.

LIST NEW MESSAGES

```
GET https://graph.microsoft.com/v1.0/users/{mailbox}/mailFolders/Inbox/messages

{
  "$top": 50,
  "$orderby": "receivedDateTime asc",
  "$select": "id,internetMessageId,receivedDateTime,subject,from,body,hasAttachments",
  "$filter": "receivedDateTime ge {watermark}"
}
```

`ge` and not `gt`: the watermark is deliberately backdated and the seen-set is what prevents reprocessing. Listing a message twice costs one skipped row; missing one loses it for good.

THEN, PER MESSAGE

```
GET /users/{mailbox}/messages/{id}/attachments
GET /users/{mailbox}/messages/{id}/attachments/{attachmentId}/$value
```

Two traps that cost us records. `hasAttachments` reads **false** for a message whose only images are inline, so a pasted-in photograph of a delivery note is invisible if you trust it. And an `itemAttachment` — a forwarded email, the single most common shape in Premier's mail — returns **MIME bytes, not a .msg file**; sniffing it as `.msg` emptied every POD field in 13 of 13 records.

Step 2 — parse

No HTTP except OCR. The body, the attachments and any scanned proof of delivery are read into one extracted record: PO number, spec code, quantity, UOM, delivery date, receiver. OCR is Azure Document Intelligence and is the only paid call in the pipeline:

```
POST {endpoint}/documentintelligence/documentModels/prebuilt-layout:analyze?api-version=2024-11-30&pages=1-10
Ocp-Apim-Subscription-Key: <key>
Content-Type: application/octet-stream
<raw bytes>

-> 202 Accepted, with an operation-location response header
GET {operation-location} -> poll until status leaves "running"
```

`pages=1-10` is sent only for PDFs; billing is per page and the polling GETs are free. The `pages` parameter is rejected for images.

Step 3 — PO number → project → DocMasterKey

This is the step everything else depends on, and there is **no single endpoint that does it**. An email gives us a PO number; every useful Spitfire endpoint is keyed by GUID. Three hops:

HOP 1 — WHICH PROJECTS CAN WE SEARCH?

ASK THE SERVER

```
POST /api/projects
```

```
{
  "IncludeHidden": true,
  "IncludeClosed": true
}
```

RESPONSE — HTTP 200 — 0 ROW(S) — 2 BYTES — 407 MS

```
[]
```

Zero rows. This account has no project list of its own, so the three ids come from configuration (`SPITFIRE_PROJECT_IDS`): `MRC024PB100003`, `MRC024PB100002`, `MRC026PB100002`. The live call is still attempted first, so this corrects itself the moment Premier grants project membership. The GET form is not an alternative:

AND THE OTHER VERB

```
GET /api/projects
```

RESPONSE — HTTP 405 — 72 BYTES — 250 MS

```
{"Message":"The requested resource does not support http method 'GET'."}
```

HOP 2 — SEARCH EACH PROJECT FOR THE PO NUMBER

THE SEARCH THAT FINDS IT

```
POST /api/project/MRC024PB100003/docs
```

```
{
  "DocNoLike": "212559",
  "IncludeDocs": true,
  "IncludeFiles": false,
  "IncludeClosed": true,
  "ResultLimit": 25,
  "ForDocType": "ff1975fd-76de-486c-888b-54e8fcd880e0"
}
```

RESPONSE — HTTP 200 — 1 ROW(S) — 2012 BYTES — 577 MS

```
[{"DocMasterKey":"1023ab34-f62c-4a2f-9d73-4c73446d3473","DocTypeKey":"ff1975fd-76de-486c-888b-54e8fcd880e0","DocReference":"00000000-0000-0000-0000-000000000000","DocDate":"2025-12-29T00:00:00","DocNo":"212559","DocBatchNo":"?","SourceDocNo":"","Title":"PO 212559 Accessories","Priority":5,"Confidential":false,"FromUser":"99cea406-bcbf-4542-8ca6-046ae7c4b1f8","SortFrom":"Pabon","ResponsibleParty_dv":"Alejandra Pabon","ResponsibleParty":"99cea406-bcbf-4542-8ca6-046ae7c4b1f8","Company":"6f93577f-1525-496d-8dda-84930fedd475","Company_dv":"Pigeon & Poodle","ToUser":"Pigeon & Poodle","SortTo":"Pigeon & Poodle","Author":"Alejandra Pabon","Due":null,"Signoff":"2026-01-26T16:56:52.78","Closed":null,"Project":"MRC024PB100003","Specification":null,"SubContract":"212559","PayControl":"","Subtype":"P 05","Subtype_dv":"SUA","Status":"M","Status_dv":"Committed","CostImpact":1916.9700,"csAmount":0.0000,"Bond":0.0000,"Subsegment":"?","ContractType":"FPR","SourceDate":"2026-02-05T00:00:00","PayItemNumber":null,"ExternalDocNo":null,"SourceContact_dv":"Pigeon & Poodle","SourceContact":"6f93577f-1525-496d-8dda-84930fedd475","Section":null,"DraftNumber":null,"Area":0,"csDate":"0001-01-01T00:00:00","DaysRequested":0,"csQty":0.0,"csNumber":0,"Probability":0,"Final":false,"Reason":null,"DocEdit":true,"SOVItemNumber":"","?","BudgetRevFlag":null,"InPeriod":"202512","OwnerApprover":"00000000-0000-0000-0000-0000 ... truncated; full response in prd_endpoint_responses.json
```

The other two answer `[]` for the same body — that is how the PO's project is discovered, by elimination:

Project searched	Rows
MRC024PB100003	1 — found
MRC024PB100002	0
MRC026PB100002	0

`DocNoLike` is a *contains* match, so `2124` would return 212456 alongside 212457. Every candidate is re-checked for an exact hit on `DocNo` or `SubContract` before its key is accepted.

HOP 3 — THE PROJECT CODE COMES FROM THE PO HEADER, NOT THE SEARCH

```
GET /api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473
-> header.Project = the project code
-> header.SubContract = the PO number, which is the join to everything else
```

All of it is then cached in `spitfire_po_index` keyed on the PO number, so a known PO costs no search at all. **If the mirror has never seen a PO, posting is refused** — there is no `forProject` without it, and the receipt cannot be created.

There is a documented fallback — a site-wide catalog search — and it has **never once executed**. It is called with `DocNoLike` and demands `TitleLike`, so it 400s and the exception is swallowed:

THE DEAD FALLBACK

POST /api/catalog/search/0/contents

```
{
  "DocNoLike": "212559",
  "IncludeDocs": true,
  "IncludeFiles": false,
  "IncludeClosed": true,
  "ResultLimit": 25,
  "ForDocType": "ff1975fd-76de-486c-888b-54e8fcd880e0"
}
```

RESPONSE — HTTP 400 — 75 BYTES — 280 MS

```
{"ThisStatus":400,"ThisReason":"catalogFilters.TitleLike requires a value"}
```

Step 4 — read the purchase order

THE HEADER — ABOUT 120 FIELDS

```
GET /api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473
```

RESPONSE — HTTP 200 — 3955 BYTES — 422 MS

```
{
  "DocMasterKey": "1023ab34-f62c-4a2f-9d73-4c73446d3473",
  "DocTypeKey": "ff1975fd-76de-486c-888b-54e8fcd880e0",
  "DocTypeKey_dv": "PO/Contracts",
  "DocReference": "00000000-0000-0000-0000-000000000000",
  "DocDate": "2025-12-29T00:00:00",
  "DocNo": "212559",
  "SourceDocNo": null,
  "ExternalDocNo": null,
  "DocBatchNo": null,
  "Title": "PO 212559",
  "Accessories": "32",
  "Priority": 5,
  "NumToSend": 0,
  "NumToForward": 0,
  "Probability": 0,
  "AutoTitled": false,
  "Confidential": false,
  "DocEdit": true,
  "Final": false,
  "DocFlag": false,
  "Due": "0001-01-01T00:00:00",
  "Closed": "0001-01-01T00:00:00",
  "Signoff": "2026-01-26T16:56:52.78",
  "SourceDate": "2026-02-05T00:00:00",
  "LinkedDocKey": "00000000-0000-0000-0000-000000000000",
  "UniReferenceKey": "00000000-0000-0000-0000-000000000000",
  "ResponsibleParty": "00000000-0000-0000-0000-000000000000",
  "ResponsibleParty_dv": "",
  "SourceContact": "6f93577f-1525-496d-8dda-84930fedd475",
  "OwnerApprover": "00000000-0000-0000-0000-000000000000",
  "LastStatusBy": "99cea406-bcbf-4542-8ca6-046ae7c4b1f8",
  "LastRouteKey": "9e0a0e57-97f8-40f0-9a2a-5ed33cad4c64",
  "Status": "M",
  "Status_dv": "Committed",
  "StatusChoices": [
    {
      "key": null,
      "label": "Canceled",
      "value": "C"
    },
    {
      "key": null,
      "label": "Committed",
      "value": "M"
    },
    {
      "key": null,
      "label": "Completed",
      "value": "D"
    }
  ],
  "Subtype": "POS",
  "ContractType": "FPR",
  "Reason": null,
  "Area": 0,
  "Duration": 0,
  "UpdateMask": 0,
  "UpdateMaskChoices": [
    {
      "key": null,
      "label": " ( No Targets )",
      "value": "0"
    },
    {
      "key": null,
      "label": " ",
      "value": " "
    }
  ],
  "truncated": true,
  "full response in prd_endpoint_responses.json"
}
```

Of those we keep DocNo , SubContract , Project , Status and DocDate . DocDate is the order date, established by testing all three candidate date fields against PO numbering across 27 consecutive pairs: DocDate got 0 out of order, SourceDate inverted on 11. /api/document/{id}/dates sounds right and is not — it returns schedule rows keyed by an unnamed GUID and carries no order date, so it is never called.

THE LINES — 68 FIELDS EACH

```
GET /api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473/items
```

RESPONSE — HTTP 200 — 6 ROW(S) — 49420 BYTES — 1812 MS

```
[
  {
    "DocItemKey": "6b3e5233-436e-42bd-9030-1f85f12f27d5",
    "TaskCount": 1,
    "CommentCount": 0,
    "LinkCount": 0,
    "ItemRevisionMap": {
      "DocRevItemKey": "6b3e5233-436e-42bd-9030-1f85f12f27d5",
      "DocItemKey": "6b3e5233-436e-42bd-9030-1f85f12f27d5",
      "ContainerKey": "00000000-0000-0000-0000-000000000000",
      "FromUser": "99cea406-bcbf-4542-8ca6-046ae7c4b1f8",
      "ItemNumber": "0001",
      "ItemSeq": 20,
      "Created": "2026-01-05T08:52:50.31",
      "ETag": "da40b18d1d6cfff19aa16619c0de258f7",
      "DocItemTask": [
        {
          "ItemTaskKey": "a124b3ba-524d-4604-a2fe-1ccfd882e33f",
          "LinkedLineKey": "00000000-0000-0000-0000-000000000000",
          "LinkedRFQKey": "00000000-0000-0000-0000-000000000000",
          "LinkedCCKKey": "00000000-0000-0000-0000-000000000000",
          "ProjEntity": "102011215",
          "ProjEntityDescription": "Accessories",
          "AccountCategory": "MAT-FDP",
          "SubContract": "",
          "SubChangeOrder": null,
          "CostType": "SP",
          "GLAcct": "900001.FDP",
          "GLSub": "MRC",
          "LaborClass": null,
          "UOM": "Set",
          "RetentionMethod": null,
          "Vendor": null,
          "ProjectReference": "MRC024PB10003",
          "ItemPercent": 0,
          "Quantity": 4.000,
          "RevenueAmount": 536.0000,
          "Rate": 134.0000,
          "ExpenseAmount": 536.0000,
          "StoredAmount": 0.0000,
          "WorkAmount": 536.0000,
          "RetentionAmount": 0.0000,
          "SMRetentionAmount": 0.0,
          "SOVWork": 0.0,
          "SOVMaterials": 0.0,
          "MarkupRate": 0.0,
          "csAmount": 0.0,
          "csValue": 0.0,
          "csQty": 0.0,
          "csNumber": 0,
          "MarkupControl": true,
          "csCheck": false,
          "csFlag": false,
          "Note": "delivered on 1/20 under FedEx 397910419278 - Pending receiver",
          "csNote": null,
          "csCode": null
        }
      ]
    }
  ],
  "truncated": true,
  "full response in prd_endpoint_responses.json"
]
```

Quantities are nested in RelatedLineDetails ; the cost code and UOM are in DocItemTask[0] . Two fields are easily confused and are not the same thing:

Field	Value on line 0001	What it is
DocItemTask[0].ProjEntity	102011215	the cost code — what the receipt line posts against
RelatedLineDetails.GLAcct / GLSub	900001.FDP / MRC	the general-ledger account. Not used by us.
DocItemTask[0].AccountCategory	MAT-FDP	what makes a line receivable. TAX- and FRT- prefixes are skipped.

On corpus PO 208491, 10 of 25 lines are non-receivable tax and freight, and nothing about their shape distinguishes them from an under-populated goods line — the account category is the only reliable discriminator.

VENDOR AND SHIP-TO

```
GET /api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473/addresses
```

RESPONSE — HTTP 200 — 2 ROW(S) — 5389 BYTES — 281 MS

```
[
  {
    "DocAddrKey": "e49a5b2e-8868-4b56-ab13-d21a84e1fa18",
    "AddrType": "F",
    "SourceType": "U",
    "UseSource": false,
    "UserKey": "99cea406-bcbf-4542-8ca6-046ae7c4b1f8",
    "Person": "Alejandra Pabon",
    "Company": "",
    "Addr1": "14185 Dallas Parkway",
    "Addr2": "Suite 1150",
    "City": "Dallas",
    "State": "TX",
    "Zip": "75254",
    "Phone": null,
    "Fax": null,
    "Email": "alejandrapabon@premierpm.com",
    "ContactProject": "MRC024PB10003",
    "RoleName": "",
    "Title": "",
    "ETag": "9f8889e21e8b15d00b2bbeda15a8c464",
    "MenuCommands": [
      {
        "MenuID": "",
        "CommandName": "OpenMap",
        "CommandArgument": null,
        "Enabled": true,
        "HasPermits": 0,
        "IconImageUrl": "fa-regular fa-map",
        "ItemText": "Show Map",
        "InfoText": null,
        "DefaultValue": null,
        "HRef": null,
        "HRefTarget": null,
        "UCModule": "DOC",
        "UCFunction": "Addr",
        "NeedPermits": 1,
        "MenuSeq": 5,
        "Confirm": null,
        "Choices": null,
        "Items": null,
        "HideifDisabled": false
      },
      {
        "MenuID": "",
        "CommandName": "UseProject",
        "CommandArgument": null,
        "Enabled": true,
        "HasPermits": 0,
        "IconImageUrl": "/Training/images-19/32/building.gif",
        "ItemText": "Use Project Setup",
        "InfoText": null,
        "DefaultValue": null,
        "HRef": null,
        "HRefTarget": null,
        "UCModule": "DOC",
        "UCFunction": "Addr",
        "NeedPermits": 4,
        "MenuSeq": 8,
        "Confirm": null,
        "Choices": null,
        "Items": null,
        "HideifDisabled": true
      },
      {
        "MenuID": "",
        "CommandName": "UseContact",
        "CommandArgument": null,
        "Enabled": true,
        "HasPermits": 0,
        "IconImageUrl": "/Training/images-19/32/building.gif",
        "ItemText": "Change Contact",
        "InfoText": null,
        "DefaultValue": null,
        "HRef": null,
        "HRefTarget": null,
        "UCModule": "DOC",
        "UCFunction": "Addr",
        "NeedPermits": 4,
        "MenuSeq": 11,
        "Confirm": null,
        "Choices": null,
        "Items": null,
        "HideifDisabled": true
      }
    ]
  },
  {
    "DocAddrKey": "e49a5b2e-8868-4b56-ab13-d21a84e1fa18",
    "AddrType": "S",
    "SourceType": "U",
    "UseSource": false,
    "UserKey": "99cea406-bcbf-4542-8ca6-046ae7c4b1f8",
    "Person": "Alejandra Pabon",
    "Company": "",
    "Addr1": "14185 Dallas Parkway",
    "Addr2": "Suite 1150",
    "City": "Dallas",
    "State": "TX",
    "Zip": "75254",
    "Phone": null,
    "Fax": null,
    "Email": "alejandrapabon@premierpm.com",
    "ContactProject": "MRC024PB10003",
    "RoleName": "",
    "Title": "",
    "ETag": "9f8889e21e8b15d00b2bbeda15a8c464",
    "MenuCommands": [
      {
        "MenuID": "",
        "CommandName": "OpenMap",
        "CommandArgument": null,
        "Enabled": true,
        "HasPermits": 0,
        "IconImageUrl": "fa-regular fa-map",
        "ItemText": "Show Map",
        "InfoText": null,
        "DefaultValue": null,
        "HRef": null,
        "HRefTarget": null,
        "UCModule": "DOC",
        "UCFunction": "Addr",
        "NeedPermits": 1,
        "MenuSeq": 5,
        "Confirm": null,
        "Choices": null,
        "Items": null,
        "HideifDisabled": false
      },
      {
        "MenuID": "",
        "CommandName": "UseProject",
        "CommandArgument": null,
        "Enabled": true,
        "HasPermits": 0,
        "IconImageUrl": "/Training/images-19/32/building.gif",
        "ItemText": "Use Project Setup",
        "InfoText": null,
        "DefaultValue": null,
        "HRef": null,
        "HRefTarget": null,
        "UCModule": "DOC",
        "UCFunction": "Addr",
        "NeedPermits": 4,
        "MenuSeq": 8,
        "Confirm": null,
        "Choices": null,
        "Items": null,
        "HideifDisabled": true
      },
      {
        "MenuID": "",
        "CommandName": "UseContact",
        "CommandArgument": null,
        "Enabled": true,
        "HasPermits": 0,
        "IconImageUrl": "/Training/images-19/32/building.gif",
        "ItemText": "Change Contact",
        "InfoText": null,
        "DefaultValue": null,
        "HRef": null,
        "HRefTarget": null,
        "UCModule": "DOC",
        "UCFunction": "Addr",
        "NeedPermits": 4,
        "MenuSeq": 11,
        "Confirm": null,
        "Choices": null,
        "Items": null,
        "HideifDisabled": true
      }
    ]
  }
],
  "truncated": true,
  "full response in prd_endpoint_responses.json"
]
```

AddrType : T vendor, S ship-to, F author, R remit-to.

Step 5 — mirror it locally

Reading a PO live costs four round trips, and about **8.6 seconds** even with a cached document key — verifying 29 records live is roughly 145 seconds, which is not a page render. So every PO read is written to `spitfire_po_index` (one row per PO) and `spitfire_po_lines` (one per line, keyed on Spitfire's own `DocItemKey`). The mirror is what the Records page and the posting gate both read.

Step 6 — verify

Re-reads the PO and puts its figures beside the email's. **It states the numbers and gives no verdict** — that is deliberate, and it is why a separate module decides whether a post may proceed. A reviewer can also pick a different PO line here, and that choice is persisted.

Step 7 — decide

Nine gates, each refusing by default and each naming the two facts that disagree. No HTTP of its own beyond the verify read.

#	Gate	Refusal it produces
1	record complete	"the record is incomplete — missing: received-by"
2	POD bytes in hand	"the delivery email carried no attachments..."
3	not already posted	"this delivery was already posted as receipt 0004"
4	PO found in our projects	"not found in the projects this connector can search"
5	line resolved by spec, or chosen by a person	"matched on description alone"
6	something still outstanding	"shows nothing outstanding on this line (19 of 19)"
7	quantity within outstanding	"this would over-receive: the email says 40 Set and only 2..."
8	units agree	"the email says CS and the purchase order says EA"
9	project known locally	"the project... is not known locally — verify the PO once"

A partial delivery is **not** a discrepancy: the comparison is against what is still outstanding, so 2 arriving against 19 ordered posts and leaves 17 outstanding. Only exceeding the outstanding quantity needs a person.

Step 8 — post the proof of delivery

Five calls, and a read-back after each thing that matters. Written to our ledger the moment each succeeds, because there is no transaction and no way to ask afterwards what happened.

8.1 CREATE THE RECEIPT — NO BODY AT ALL

```
POST /api/document/00000000-0000-0000-0000-000000000000/{receiptTypeKey}?forProject={projectCode}&forBatch={poNumber}
-> 200, a BARE QUOTED GUID, not an object:
"2189d7d7-da0b-47f0-919b-0c3a2b07d6ce"
```

The parent is the null GUID and the type key is the *second path segment*. `forBatch` is the PO number and is what populates the header's `SubContract` — **the only field tying a receipt to its purchase order**. If it does not come back set, the receipt is an orphan and nothing further is attached.

Creating the receipt stages people. Spitfire applies the configured approval chain the instant the document exists: six routees, three of them real Premier employees at sequence 10. Nothing is emailed — that needs `route/apply`, which the write connector refuses by name — but they are on it.

8.2 TITLE IT — THE BODY IS A BARE JSON STRING

```
PATCH /api/document/{receiptKey}/Title
"CC-TEST - receiver automation - PO 212559 - 2026-08-17 - DO NOT PROCESS"
```

8.3 ADD THE RECEIPT LINE — THE BODY IS AN ARRAY

```
[
  {
    "Description": "CC-TEST Amenity Tray at Ballroom Restrooms",
    "ItemQuantity": 4.0,
    "DocItemTask": [
      {
        "Quantity": 4.0,
        "UOM": "Set",
        "ProjEntity": "102011215"
      }
    ],
    "SourceItemNumber": "BRR-803-AC"
  }
]
```

PUT /items is an undiscoverable 500; POST is the working call. UOM and ProjEntity are copied from the matched PO line rather than invented — a wrong ProjEntity books the receipt to the wrong budget. Lines read back as {DocNo}-{seq}, never as the integer sent.

8.4 UPLOAD THE POD — MULTIPART, TWO PARTS, NO CONTENT-TYPE HEADER

```
POST /api/catalog/upload?xm=catalog
  part 1 fileMeta (application/json)
  part 2 file      (application/octet-stream)
```

```
-> 200 {"name":"CC-TEST.POD.FedEx.pdf","key":"8a0af8b5-3fb3-44e5-af75-b4ebd490fcca","size":64,"progress":"100.0%","error":null}
```

```
{
  "value": "CC-TEST.POD.FedEx.pdf",
  "Name": "CC-TEST.POD.FedEx.pdf",
  "type": "file",
  "FileType": "pdf",
  "size": 20535,
  "MD5": "8342F644E7DC630772686415F287020B",
  "date": "2026-08-17T00:00:00",
  "DocDate": "2026-08-17T00:00:00",
  "ReferenceDate": "2026-08-17T00:00:00",
  "Due": "2026-08-17T00:00:00",
  "Keywords": "CC-TEST POD 212559"
}
```

The failure ladder, each rung walked into in turn:

What is wrong	What the server says
no file part, or a hand-set Content-Type	415 "File expected" — setting the header by hand loses the multipart boundary
no fileMeta part	400 "Meta data missing"
filemeta in lower case	400 — part names are case-sensitive
any of the four dates missing	500 "Nullable object must have a value"

MD5 must be upper case. Spitfire takes the stored filename from the file part and ignores fileMeta.Name.

8.5 VERIFY THE BYTES, THEN ATTACH

```
GET /api/catalog/{fileKey}/versions
-> [{ ..., "DataHash": "8342F644E7DC630772686415F287020B" }]
the server's own MD5 — the only real proof the bytes arrived intact.
/meta and /object return 500 on the very same key /versions accepts.
```

```
[
  {
    "DocKey": "<fileKey>",
    "AttachedDocMaster": "00000000-0000-0000-0000-000000000000",
    "Note": "CC-TEST proof of delivery",
    "CatType": "<receipt doc type>",
    "MailRoute": "P",
    "AccessLevel": "V"
  }
]
```

```
-> 200 {"DocAttachKey":"0bc8af59-0bd2-4f90-80cb-ada94c10fffe","DocKey":"8a0af8b5-..."}
```

Step 9 — post the receiver report

A separate decision a person makes, so it is a separate button and a separate ledger state. The report is generated per delivery, rendered to PDF through headless Chrome, then uploaded and attached exactly as the POD was. Two document links follow.

One endpoint, two body shapes. A file link populates `DockKey` ; a document link populates `AttachedDocMaster` and leaves `DockKey` as the null GUID. Premier's own receipt 209330 carries both shapes in one collection.

```
[
  {
    "DockKey": "00000000-0000-0000-0000-000000000000",
    "AttachedDocMaster": "<the PO's DocMasterKey>",
    "Note": "CC-TEST purchase order",
    "CatType": "<PO doc type>",
    "MailRoute": "P",
    "AccessLevel": "V"
  }
]
```

Not `PUT /api/document/{id}/link` , which *creates* child documents from type keys and returns 500 when handed an existing key.

Pay requests are found by the same `SubContract` join and linked the same way. This link matters more than it looks: a receiver without it leaves reporting treating the item as never received, so depreciation never starts — the gap behind Premier's manual quarterly catch-up.

Step 10 — what Spitfire shows afterwards

Nothing, until a human approves the receipt. It is left *In Process* and is never routed, so the purchase order goes on reporting nothing received. That is by design — dispatching the approval chain emails real Premier staff, and that is Premier's decision, not a button's.

Reference A — every endpoint, and what it answered

Captured 2026-08-18 against `https://training.remingtonhotels.com/Training` . Regenerate with `tools/verify_prd_endpoints.py` .

Method	Path	Status	Rows	Why we call it
GET	/api/system/version	200	—	anonymous; proves reachability
GET	/api/account/session	200	—	session probe
GET	/api/session/who	200	—	whose ticket this is
GET	/api/projects	405	—	cannot enumerate projects; documented 405
POST	/api/projects	200	0	the POST form; why the three project ids are configuration, not discovery
POST	/api/project/MRC024PB100003/docs	200	1	search MRC024PB100003 for PO 212559
POST	/api/project/MRC024PB100002/docs	200	0	search MRC024PB100002 for PO 212559
POST	/api/project/MRC026PB100002/docs	200	0	search MRC026PB100002 for PO 212559
POST	/api/catalog/search/0/contents	400	—	resolve_po's fallback — never actually executes
GET	/api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473	200	—	PO header
GET	/api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473/items	200	6	PO lines
GET	/api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473/addresses	200	2	vendor and ship-to
GET	/api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473/route	200	5	purchasing agent
GET	/api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473/attachments	200	12	what hangs off the PO
GET	/api/configuration/doctypes	404	—	would list document type GUIDs
GET	/api/configuration/documenttypes	404	—	would list document type GUIDs

GET	/api/configuration/doctype	404	—	would list document type GUIDs
GET	/api/config/doctypes	500	—	would list document type GUIDs
GET	/api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473/dates	200	1	allowlisted but deliberately never called
GET	/api/document/00000000-0000-0000-0000-000000000001	500	—	an invented document GUID
GET	/api/document/1023ab34-f62c-4a2f-9d73-4c73446d3473/items	200	6	re-read to measure ReceiptInProgressUnits against known receipts

Reference B — endpoints that answer nothing useful

Every one of these was called while writing this document. They are listed so nobody spends an afternoon rediscovering them.

Endpoint	What happens	What we do instead
POST /api/projects	200 with zero rows	the three project ids are configuration, not discovery
GET /api/projects	405	—
GET /api/configuration/*	404 — no such controller on this build	document-type GUIDs are hardcoded from Premier's own SQL
GET /api/config/doctypes	500	as above
POST /api/catalog/search/0/contents	400 "catalogFilters.TitleLike requires a value"	dead fallback; the project-scoped search is what works
GET /api/document/{unknown-guid}	500, not 404 — and byte-identical to a genuine fault	a 500 can never be read as "not found"
GET /api/document/{id}/dates	200, but schedule rows keyed by an unnamed GUID	the order date is the header's DocDate
PUT /api/document/{id}/items	500	POST, with an array body
PUT /api/document/{id}/link	500 when handed an existing key	document links go through /attachments
GET /api/catalog/{key}/meta and /object	500 on a key /versions accepts	/versions

Reference C — the GUIDs we hardcode, and why

The endpoint that would list document types does not exist, so these come from Premier's own `czx_TPICreate_ReceiptDoc.sql` and from reading `xsFDocHeader` directly.

Constant	Value	Provenance
purchase order type	ff1975fd-76de-486c-888b-54e8fcd880e0	@PODTK in Premier's SQL. The master plan transcribes this one character short.
receipt type	0c9a537a-3c41-4d16-ab9f-130ef69ea6c8	confirmed against TrainingsfDocSys: 76,414 documents carry it, DocTypeKey_dv reads "Receipt"
pay request type	5b0a71d8-ed55-455b-bb99-2ab7d3b7a1cf	@PRDTK in the same SQL
null GUID	00000000-0000-0000-0000-000000000000	Spitfire's "no value"; the parent slot on create and the unused half of an attachment row
project ids	MRC024PB100003, MRC024PB100002, MRC026PB100002	read out of xsFDocHeader ; 18 / 8 / 2 corpus POs respectively

Reference D — corrections this exercise produced

Five things the codebase or an earlier report stated that the server contradicts. All five were checked by calling the endpoint.

Was documented as	Actually	Consequence
<code>/api/configuration/*</code> returns 500 "not yet implemented (case 36629)"	404 — "No type was found that matches the controller named 'configuration'"	the controller does not exist; case 36629 may be chasing something that was never there
the site-wide catalog search "answers 200 with zero rows"	400 — it requires <code>TitleLike</code> and we send <code>DocNoLike</code>	the fallback in <code>resolve_po</code> has never executed; it throws and is swallowed
"re-sending an attach creates a second row"	400 "This file is already attached" for the same fileKey	the no-retry rule is still right, but for a different reason: a retried <i>upload</i> mints a new fileKey, and that one attaches cleanly
<code>AttachNote</code> carries the attachment note	silently ignored — returns 200 , reads back <code>Note: null</code> . <code>Note</code> is the field that works	the old write probe recorded notes that were never stored
<code>CatType</code> groups attachments by category	confirmed dropped — a valid receipt GUID was sent and <code>00000000-...</code> stored	attachments cannot be grouped by category; do not build on it

Reference E — auth, in one paragraph

Spitfire has no `Authorization` header. A single cookie, `sfPMSAuth`, is the FormsAuthentication ticket and is sufficient on its own; the other three cookies a browser holds are a session id, settings and a session GUID, none of which authenticate. It is copied by hand from a browser because Premier's Spitfire has Entra SSO, and it **lapses on idle** — which is why an expired ticket raises a named error rather than being silently re-authenticated. Every write is recorded by the ERP as `api@consciouscreations.ai` regardless of who triggered it, so our own audit log is the only record that can tell two operators apart. **Graph** is unrelated: MSAL client-credentials, a bearer token, app-only permissions consented on the app registration.